Project 2. CSR SpMV: Profiling Access Locality Impact with Hotspot Analysis and Working Set Sweep

1. Reproduction Flow

```
$ ./run.sh
```

Running `./run.sh` collects system information, builds the benchmark, runs the self-test and hotspot analysis, executes the smoke test and full working-set sweep, and generates the SVG plot and source checksums.

2. System Information

The script inspects `lscpu`, `/proc/sys/kernel/perf_event_paranoid`, relevant `perf list` cache and TLB events, and the detailed `perf list --details cache-misses` output.

Field	Value
Model name	Intel Xeon Gold 6526Y
BIOS model name	Intel Xeon Gold 6526Y
L1d cache	1.5 MiB total (48 KiB per core, 32 instances)
L2 cache	64 MiB total (2 MiB per core, 32 instances)
L3 cache	75 MiB (2 instances)
NUMA nodes	2
NUMA node 0 CPUs	0–15, 32–47
NUMA node 1 CPUs	16–31, 48–63

On Intel Emerald Rapids (Intel's 5th-generation Xeon Scalable server-processor architecture), the benchmark uses the L2 event `L2_RQSTS.DEMAND_DATA_RD_MISS`, encoded by the raw value `0x2124`. It counts true demand data-read misses in the L2 cache and does not count misses that were merged with other requests.

3. Hotspot Analysis

Linux `perf` samples CPU activity and reports where execution time is spent. The automated workflow profiles the permuted SpMV case with `perf record`, then uses `perf report` and `perf annotate` to inspect the profile.

Command	Purpose
`perf record`	Collects sampled execution data while the benchmark runs and saves it as a profile.
`perf report`	Summarizes the collected profile to show where execution time was spent.
`perf annotate`	Associates sampled execution data with source or assembly lines to identify hotspots in detail.

The profile was collected for the entire process using the user-space `cycles:u` event. It captured 9,115 samples with no lost samples. `main` accounts for 5.11% of self time and 96.92% of child time, while the `csr_spmv` kernel accounts for 91.68% of self time and 91.65% of child time. The kernel is therefore the primary hotspot rather than setup or result verification.

The annotated kernel contains the inner accumulation loop `sum += val[k] * x[ci[k]]`. The floating-point multiply instruction accounts for 78.74% of samples, followed by the indexed `x[ci[k]]` load at 3.95%, loop-bound comparison at 4.31%, and loop increment at 8.18%. This identifies the multiply and indirect vector access inside `csr_spmv` as the dominant sampled work.

4. Experiment

4.1 Self Test and Smoke Test

The self-test and smoke test both pass.

4.2 Working-Set Sweep

For each case, the runner starts a separate benchmark process with `--pmu` and another with `--pmu-levels`, checks that both results are correct and have matching checksums, and then combines the measurements into the experiment summary. The two PMU modes run sequentially rather than simultaneously, so the core's programmable-counter limit is not exceeded.

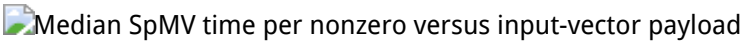
The sweep changes the sparse-matrix row count while keeping `nnz-per-row=8` and $C = 4R$. Because each element of the dense input vector x is an 8-byte double, the input payload is $8C = 32R$ bytes. The selected sizes span the per-core L2 and socket-level LLC capacities reported for the target server:

R	x payload	Reason to include
16,384	0.5 MiB	Smaller than one core's L2
65,536	2 MiB	Near one core's L2
262,144	8 MiB	Beyond L2; below socket LLC
1,048,576	32 MiB	Near socket LLC capacity
2,097,152	64 MiB	Above socket LLC capacity

The `local` and `permuted` layouts are both tested at every size. The payload values describe only the input vector x; the CSR arrays, output vector, and correctness oracle add additional working-set data. The runner uses 400, 300, 100, 40, and 20 timed iterations for the five sizes respectively, with five repetitions per layout by default.

The L2 event is collected only after the benchmark identifies an Emerald Rapids CPU, and Linux must also permit access to it. Permissions, kernel support, or PMU availability may prevent collection; if the L2 event is unavailable, its result is reported as `N/A`, the experiment continues with any other available counters, and `PMU_STATUS=PARTIAL` is acceptable when only some fields are missing. Consult `PMU_UNAVAILABLE_EVENTS` for failure details and record the CPU model. For meaningful comparisons, report per-iteration or per-nonzero normalized counts within the same PMU mode, since different layouts can reverse rankings at individual cache levels.

5. Experiment Result



The plot shows the median normalized kernel time from the full working-set sweep. The horizontal axis is the input-vector payload, and the vertical axis is median nanoseconds per nonzero. The table below keeps the measurements most relevant to the working-set and locality analysis. PMU counts are medians from the corresponding five repetitions; all rows passed correctness and had five successful core and levels PMU passes.

5.1 Local layout

R	x payload	Median ms/iter	Median ns/nonzero	Median IPC	L1D load misses	L2 demand-read misses	LLC load misses	dTLB load misses
16,384	0.5 MiB	0.07	0.53	4.20	3,342,792	27,973	0	0
65,536	2 MiB	0.38	0.73	3.03	8,008,429	56,810	11	14
262,144	8 MiB	1.79	0.85	2.62	8,714,371	40,195	426,525	447
1,048,576	32 MiB	10.62	1.27	1.75	10,988,033	53,125	4,601,925	245
2,097,152	64 MiB	23.60	1.41	1.58	10,476,474	57,985	5,831,429	157

For the local layout, median ns/nonzero rises from 0.53 at 0.5 MiB to 1.41 at 64 MiB, a 2.70x increase. The increase is gradual: performance remains below 0.90 ns/nonzero through the 8 MiB case, then rises to 1.27 and 1.41 ns/nonzero as the payload approaches and exceeds socket LLC capacity. IPC decreases from 4.20 to 1.58 across the same range, while L1D load misses remain between roughly 3.34 and 10.99 million. This is the expected behavior for the layout that preserves local access order as the input payload moves through the cache hierarchy.

5.2 Permuted layout

R	x payload	Median ms/iter	Median ns/nonzero	Median IPC	L1D load misses	L2 demand-read misses	LLC load misses	dTLB load misses
16,384	0.5 MiB	0.09	0.67	3.34	50,227,510	21,987	0	1
65,536	2 MiB	0.68	1.30	1.71	151,573,233	33,708,556	84	320
262,144	8 MiB	4.95	2.36	0.94	157,229,227	76,778,411	4,517,464	1,190
1,048,576	32 MiB	31.06	3.70	0.60	243,254,149	144,786,872	52,767,638	1,004
2,097,152	64 MiB	79.64	4.75	0.47	220,871,734	100,414,079	182,387,304	4,104

For the permuted layout, median ns/nonzero rises from 0.67 at 0.5 MiB to 4.75 at 64 MiB, a 7.14x increase. The sharpest transition is from 2 MiB to 8 MiB, where the payload moves from near one core's L2 to beyond L2. IPC falls from 3.34 to 0.47, and the L1D load-miss count reaches 243,254,149 at 32 MiB. At 64 MiB, LLC load misses reach 182,387,304 and dTLB load misses reach 4,104, showing that the irregular access pattern continues to stress lower cache levels and address translation as the working set grows.

5.3 Local vs Permuted Layout

The working-set design uses payloads of 0.5, 2, 8, 32, and 64 MiB. These are, respectively, below one core's 2 MiB L2, near L2, beyond L2 but below the 75 MiB socket LLC, near the LLC capacity, and above the LLC capacity. Because

`nnz-per-row=8` remains fixed and only R changes, the sweep isolates the effect of increasing the input vector's working set while comparing two access layouts.

At every size, the permuted layout is slower than the local layout. The penalty grows from 1.26x at 0.5 MiB to 1.77x at 2 MiB, 2.77x at 8 MiB, 2.92x at 32 MiB, and 3.38x at 64 MiB. This widening separation matches the plot and connects the layout transformation to the cache-capacity transitions in the sweep-design table.

The PMU counts support this locality trend, but they are not additive traffic totals. At 32 MiB, for example, permuted layout records 243,254,149 L1D load misses, 144,786,872 L2 demand-read misses, and 52,767,638 LLC load misses, compared with 10,988,033, 53,125, and 4,601,925 for local layout. At 64 MiB, permuted LLC load misses increase to 182,387,304 while local remains at 5,831,429. These counters should be compared within the same case and PMU mode, not summed as a DRAM-traffic estimate.

All ten sweep rows have `all_correct=True`, `core_pmu_ok_count=5`, and `levels_pmu_ok_count=5`. The comparisons therefore use valid five-repetition medians rather than mixing failed runs with successful measurements.